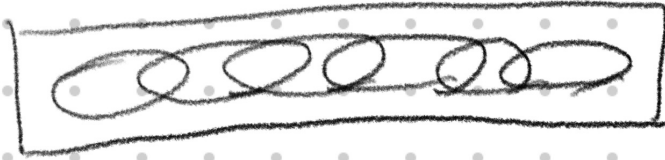


ALGORITHM	BEST CASE	AVERAGE CASE	WORST CASE
<u>Bubble Sort</u>	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
<u>Merge Sort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
<u>Heap Sort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$
<u>Quick Sort</u>	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$
<u>Counting Sort</u>	$O(n+k)$	$O(n+k)$	$O(n+k)$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$

Bubble Sort



- Basic

- $O(n)$ $\rightarrow$ $O(n^2)$
Best Worst

has_swapped = True

while has_swapped:

has_swapped = False

for i in range(len(nums) - 1):

if nums[i] > nums[i + 1]:

swap $\rightarrow$ nums[i], nums[i + 1] = nums[i + 1], nums[i]
Flagged $\rightarrow$ has_swapped = True

return nums

Optimized Bubble Sort

- If no swap $\rightarrow$ all sorted $\rightarrow$ done

$\rightarrow$ Keep swapping if nums[i] > nums[i + 1]

Merge Sort - $O(n \log n)$

- Divide and conquer

```
def merge_sort(arr):
```

```
    if len(arr) == 1: # len 1 means it is sorted
        return arr
```

Divide

```
    mid = len(arr) // 2 # Get a pivot
    left_arr = merge_sort(arr[:mid])
    right_arr = merge_sort(arr[mid:])
```

Merge

```
    return merge(left_arr, right_arr)
```

```
def merge(left_arr, right_arr):
```

```
    result = []
```

```
    i, j = 0, 0
```

```
    while i < len(left_arr) and j < len(right_arr)
```

```
        if left_arr[i] < right_arr[j]:
```

```
            result.append(left_arr[i])
```

```
            i += 1
```

```
        else:
```

```
            result.append(right_arr[j])
```

```
            j += 1
```

```
    result.extend(left_arr[i:])
```

```
    result.extend(right_arr[j:])
```

```
    return result
```

① get mid index

② divide left_arr and right_arr by mid

③ merge left_arr and right

④ compare and put smaller one in array

⑤ return all with sort

Heap Sort

- $O(n \log n)$
- in-place sort
- Max heap

```
def heap_sort(arr):
```

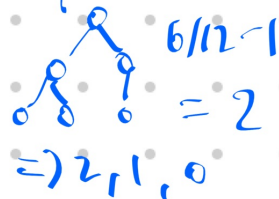
```
    n = len(arr)
```

```
    # Max heap
```

```
    for i in range(n//2 - 1, -1, -1):
```

only parent nodes

```
        heapify(n, i)
```



```
    for i in range(n-1, 0, -1):
```

Swap Root
with the last

```
        arr[i], arr[0] = arr[0], arr[i]
        heapify(i, 0)
```

```
def heapify(heap_size, index):
```

```
    left = 2 * index + 1
```

index's

left child

```
    right = 2 * index + 2
```

#

right child

```
    largest = index
```

```
    if left < heap_size and arr[left] > arr[largest]:
```

```
        largest = left
```

```
    if right < heap_size and arr[right] > arr[largest]:
```

```
        largest = right
```

```
    if largest != index:
```

```
        arr[index], arr[largest] = arr[largest], arr[index]
```

```
        heapify(heap_size, largest)
```

example:

[4, 10, 3, 5, 1] → max heap → [10, 5, 4, 3, 1] → swap root with the last
→ [1, 5, 4, 3, 10] → heapify size-1 → [5, 4, 3, 1, 10] → ...

Quick Sort

- $O(n \log n) \rightarrow O(n^2)$
- In-place
- Divide and Conquer

* Hoare Partition style

```
def quick_sort(arr, start, end):
```

```
    if start < end:
```

```
        p = hoare_partition(arr, start, end)
```

```
        quick_sort(arr, start, p) # left side include p
```

```
        quick_sort(arr, p+1, end)
```

```
def hoare_partition(arr, start, end):
```

```
    pivot = arr[start]
```

```
    left = start - 1
```

```
    right = end + 1
```

```
    while True:
```

```
        # move left pointer until arr[left] >= pivot
```

```
        while True:
```

```
            left += 1
```

```
            if left > end or arr[left] >= pivot
```

```
                break
```

```
        # move right pointer until arr[right] <= pivot
```

```
        while True:
```

```
            right -= 1
```

```
            if right < start or arr[right] <= pivot
```

```
                break
```

```
        if left >= right
```

```
            return right
```

```
        arr[left], arr[right] = arr[right], arr[left]
```